🧪 🔥 PRACTICAL 3 (ADVANCED GIT)

✅ a) Initialize Repository

```
mkdir data-science-project-3
cd data-science-project-3
git init
```

✅ b) Push to GitHub

👉 Create repo: data-science-project-3

```
git remote add origin https://github.com/your-username/data-science-project-3.git
git branch -M main
```

✅ c) Create Branch GIT-EXAM-1 + program2 (Palindrome)

```
git checkout -b GIT-EXAM-1
touch program2.py
```

Paste:

```
text = input("Enter a string: ")

if text == text[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

✅ d) Create program1.py (Squares)

```
touch program1.py
```

Paste:

```
for i in range(1, 11):
    print(i, "square is", i*i)
```

✅ e) Commit & Push

```
git add .
git commit -m "Added program1 and program2 in GIT-EXAM-1"
git push -u origin GIT-EXAM-1
```

✅ f) Create New Branch GIT-EXAM-2

```
git checkout -b GIT-EXAM-2
```

✅ g) Pull Changes from GIT-EXAM-1

```
git pull origin GIT-EXAM-1
```

👉 Now both files are available in this branch

✅ h) Create program2.py (again)

👉 (Already exists, but modify it slightly to create difference)

```
nano program2.py
```

Change it:

```
text = input("Enter string: ")

if text == text[::-1]:
    print("It is Palindrome")
else:
    print("It is Not Palindrome")
```

✅ i) Modify program1.py (FOR CONFLICT)

```
nano program1.py
```

Change code:

```
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

👉 This will create conflict later

✅ j) Commit & Push

```
git add .
git commit -m "Modified files in GIT-EXAM-2"
git push -u origin GIT-EXAM-2
```

🔍 Show Differences

```
git diff main GIT-EXAM-1
git diff GIT-EXAM-1 GIT-EXAM-2
```

👉 You will see:

- program changes
- text differences

🔥 k) CREATE PULL REQUEST (IMPORTANT)

👉 Go to GitHub

1. Go to Pull Requests
2. Click New Pull Request
3. Select:
    - base → GIT-EXAM-1
    - compare → GIT-EXAM-2

⚠️ CONFLICT WILL APPEAR (EXPECTED)

👉 GitHub may show:
"This branch has conflicts"

🔧 RESOLVE CONFLICT

Click:
👉 Resolve conflicts

You will see something like:

```
<<<<<<< HEAD
print(i, "square is", i*i)
=======
print("Square of", i, "is", i*i)
>>>>>>> GIT-EXAM-2
```

👉 Keep one version (or combine):

```
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

Click:

- Mark as resolved
- Commit merge

✅ Merge Pull Request

Click:
👉 Merge Pull Request

QNA

🧪 🔥 **WHAT THIS PRACTICAL IS (Simple Meaning)**

👉 **You used Git and GitHub to:**

- **Work on multiple branches**
- **Share code between branches (pull)**
- **Create conflict intentionally**
- **Resolve conflict manually**
- **Merge using Pull Request**

🧠 **WHAT YOU DID (Say This in Exam)**

👉
"We created a repository, worked on two branches, pulled changes from one branch to another, modified the same file to create a conflict, resolved it, and merged changes using a pull request."

🔍 **HOW IT WORKS (STEP-BY-STEP LOGIC)**

🔹 **Step 1: Branch GIT-EXAM-1**

👉 **You created:**

- **program1.py → squares**
- **program2.py → palindrome**

👉 **This is your first version**

🔹 **Step 2: Branch GIT-EXAM-2**

👉 **You:**

- **Pulled code from GIT-EXAM-1**
- **Modified files**

👉 **Now:**
➡️ **Both branches start same**
➡️ **Then become different**

🔹 **Step 3: Create Conflict**

👉 **You changed program1.py in GIT-EXAM-2**

👉 **But original version still exists in GIT-EXAM-1**

👉 **When merging → conflict happens ❌**

🔹 **Step 4: Conflict Detection**

Git shows:

```
<<<<<<< HEAD
old code
=======
new code
>>>>>>> branch
```

👉 **Means:**

- **Two versions exist**
- **Git is confused**

🔹 **Step 5: Conflict Resolution**

👉 **You manually choose:**

- **Old code OR**
- **New code OR**
- **Combine both**

🔹 **Step 6: Pull Request**

👉 **Merge:**

- GIT-EXAM-2 → GIT-EXAM-1

👉 After resolving conflict ✓

🧠 WHY WE DO THIS

---

1. 🔒 Safe Parallel Development

👉 Multiple branches allow:

- Different features
- No interference

---

2. 🔄 Real-Time Code Sharing

👉 git pull helps:

- Sync code between branches

---

3. ⚠️ Conflict Handling (VERY IMPORTANT)

👉 Real projects:

- Multiple people edit same file
- Conflicts are common

---

4. 👷 Industry Workflow

👉 This is EXACTLY how teams work:

- Branch → Work → Pull → Merge → Resolve

⭐ IMPORTANCE

✓ 1. Real Software Development Skill

👉 This is not basic Git
👉 This is real-world workflow

✓ 2. Team Collaboration

👉 Many developers working together

✓ 3. Error Prevention

👉 Conflicts must be handled properly

✓ 4. DevOps + CI/CD Base

👉 All pipelines depend on:

- Branching
- Pull request
- Merge

⚙️ COMPLETE WORKING FLOW

GIT-EXAM-1 → program1 + program2
↓
create branch
↓
GIT-EXAM-2 (pull from EXAM-1)
↓
modify same file
↓
conflict occurs
↓
resolve manually
↓
pull request
↓
merge

🧠 CORE CONCEPTS (VERY IMPORTANT)

🔹 Branch

👉 Separate version of code

🔹 Pull

👉 Bring changes from another branch/repo

🔹 **Conflict**

👉 **Same file edited in different ways**

🔹 **Conflict Resolution**

👉 **Manual correction of code**

🔹 **Pull Request**

👉 **Request to merge branches**

🔹 **Merge**

👉 **Combine final code**

🎯 **MODERATE LEVEL VIVA QUESTIONS**

❓ **What is git pull?**

👉
Git pull is used to fetch and merge changes from a remote branch into the current branch.

❓ **What is merge conflict?**

👉
A merge conflict occurs when two branches modify the same part of a file differently.

❓ **Why did conflict occur in your practical?**

👉
Because program1.py was modified differently in GIT-EXAM-1 and GIT-EXAM-2 branches.

❓ **How did you resolve conflict?**

👉
By manually editing the conflicting code and selecting the correct version.

❓ **What is difference between pull and fetch?**

👉

- **pull → fetch + merge**
- **fetch → only downloads changes**

❓ **Can pull request be created between two branches?**

👉
Yes, pull request can be created between any two branches.

❓ **What is HEAD in Git?**

👉
HEAD points to the current working branch or latest commit.

❓ **What is staging area?**

👉
Intermediate area where changes are prepared before committing.

🔥 **SLIGHTLY ADVANCED (FOR EXTRA MARKS)**

❓ **What happens if conflict is not resolved?**

👉
Merge will not complete until conflict is resolved.

❓ **What is rebase?**

👉
Rebase is used to apply changes of one branch onto another without creating a merge commit.

❓ **Why branching is important in DevOps?**

👉
It allows parallel development and smooth CI/CD pipelines.

🧠 **ONE-LINE POWER ANSWER**

👉
“This practical demonstrates advanced Git operations including multi-branch development, pulling changes, handling merge conflicts, and merging branches using pull requests.”

🏁 FINAL UNDERSTANDING

👉 This practical proves:

✓ You understand real Git workflow
✓ You can handle conflicts
✓ You can work in multi-branch environment
✓ You are ready for real-world DevOps